

Is it a view or a copy? — solutions

Answers in **red**, with a short explanation under each row. All metadata assumes `int64`, i.e. 8 bytes per item, and strides are in **bytes**.

Rule of thumb: NumPy can return a view whenever the elements you asked for sit at a *constant step* (per axis) in the existing memory block — that is what `strides` can express. Basic slicing always yields such a pattern, so it gives views; fancy indexing (integer or boolean arrays) always copies, even when the indices happen to be regularly spaced.

The one memory block behind `x` — 12 items of `int64`, 8 bytes each:

0	1	2	3	4	5	6	7	8	9	10	11
0	8	16	24	32	40	48	56	64	72	80	88

second row = byte offset of each item from the start of the block

NumPy operation	Array layout of y	NumPy array metadata	View or copy?
-----------------	-------------------	----------------------	---------------

WORKED EXAMPLE

`x`
the original array

0	1	2	3
4	5	6	7
8	9	10	11

dtype	int64
ndim	2
shape	(3, 4)
strides	(32, 8)

—

`y = x.reshape((6, 2))`

0	1
2	3
4	5
6	7
8	9
10	11

dtype	int64
ndim	2
shape	(6, 2)
strides	(16, 8)

☒ **view**
☐ copy

Why: `x` is contiguous and the elements are needed in the same order, so only the shape/strides bookkeeping changes: walk 2 items (16 bytes) per row, 1 item (8 bytes) per column.

`y = x[:, :2, :]`

0	1	2	3
8	9	10	11

dtype	int64
ndim	2
shape	(2, 4)
strides	(64, 8)

☒ **view**
☐ copy

Why: A basic slice: keep every other row by doubling the row stride to $2 \times 32 = 64$ bytes. Regular steps are exactly what strides can express.

`y = x[[1, 2, 0], [1, 1, 2]]`

5	9	2
---	---	---

dtype	int64
ndim	1
shape	(3,)
strides	(8,)

☐ view
☒ **copy**

Why: Fancy indexing picks elements at byte offsets 40, 72, 16 — not a constant step, so no single stride can reach them. NumPy must gather them into a new block.

`y = x[[0, 2], :]`

get the first and third row

0	1	2	3
8	9	10	11

dtype	int64
ndim	2
shape	(2, 4)
strides	(32, 8)

☐ view
☒ **copy**

Why: Careful: rows 0 and 2 *are* evenly spaced, so a view would be expressible here (it is what `x[:, :2, :]` returns). But this is fancy indexing, and NumPy always copies for fancy indexing — it does not inspect the index list to see whether it happens to be regular. The rule is about the *kind* of indexing, not the particular values.

`y = x.ravel()`

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

dtype	int64
ndim	1
shape	(12,)
strides	(8,)

☒ **view**
☐ copy

Why: `x` is C-contiguous, so reading the block straight through in memory order already gives the flattened array: a 1-D view with an 8-byte stride.

The one memory block behind x — 12 items of int64, 8 bytes each:

0	1	2	3	4	5	6	7	8	9	10	11
0	8	16	24	32	40	48	56	64	72	80	88

second row = byte offset of each item from the start of the block

NumPy operation

Array layout of y

NumPy array metadata

View or copy?

y = x.T

0	4	8
1	5	9
2	6	10
3	7	11

dtype	int64
ndim	2
shape	(4, 3)
strides	(8, 32)

☒ view

☐ copy

Why: Transposing just swaps the two strides: step 8 bytes down a column and 32 bytes across a row. No data moves — the result is simply non-contiguous.

y = x.T.ravel()

0	4	8	1	5	9	2	6	10	3	7	11
---	---	---	---	---	---	---	---	----	---	---	----

dtype	int64
ndim	1
shape	(12,)
strides	(8,)

☐ view

☒ copy

Why: Flattening x.T needs the elements in the order 0, 4, 8, 1, 5, 9, ... i.e. byte offsets 0, 32, 64, 8, ... That jumps backwards, so it is not a constant step and no stride can produce it. A copy is unavoidable.